DATA 266 Homework 4.5

DATA 266 — Homework 4.5

Sneha Singh
SID4 = 0670 | SEED = 670 | SLICE = 670 | HP_ID = 4 | CLS_A = 0 | CLS_B = 5

Notebook: `hw4.5.ipynb`. I ran this on an NVIDIA L4 in Colab Pro. Part C skipped (optional). One GPU.

Same SID/SEED as the earlier homework. HP_ID is reported only. I did not train a second network. Every number below is from that Colab run.

Setup

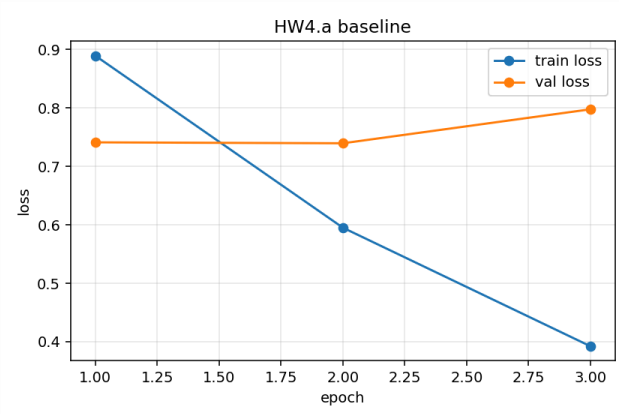
I used `cardiffnlp/tweet_eval` sentiment, train split only. Rows **670–5669** (5,000 consecutive). SEED 670 splits them **3,500 / 750 / 750**. In the slice there are 773 negative, 2290 neutral, and 1937 positive tweets. I do not use that dataset's own validation or test split.

The fine-tune is `bert-base-uncased` with BertTokenizer, max length 128, batch 16, AdamW, lr 2e-5, weight decay 0.01, 3 epochs. Serving uses `facebook/opt-125m`, 32 new tokens, greedy decoding.

HW4.a — BERT sentiment

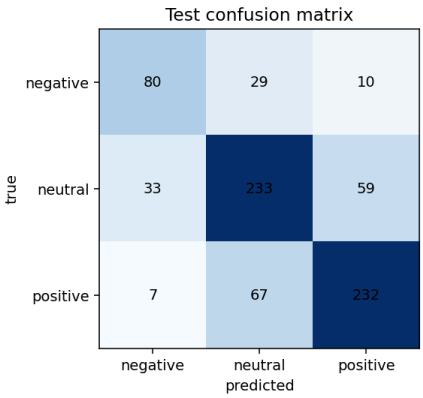
I fine-tuned BERT for three epochs. Validation loss was lowest at epoch 2 (0.7395). Epoch 3 went up to 0.7976, so I kept the epoch-2 weights for the test score. Training past that point cost me the third epoch, about **38.3 seconds**.

Epoch	Train loss	Val loss	Val accuracy	Seconds
1	0.8892	0.7410	0.6640	37.0
2	0.5950	0.7395	0.6773	37.8
3	0.3922	0.7976	0.6813	38.3



Baseline loss

Peak GPU memory for this baseline was **2.48 GB**. Wall-clock training time was **113.1 s**. Test accuracy at the best validation-loss epoch was **0.7267**.



Confusion matrix

Class	Precision	Recall	Support
negative	0.6667	0.6723	119
neutral	0.7082	0.7169	325
positive	0.7708	0.7582	306

The worst class was **negative**. That class only has 119 test examples, so some of the miss is just low support. Five negative tweets the model called neutral:

1. *“Sophos is the Lamest Virus Ever…”* — there is an insult, but the rest reads like a headline, so the model stayed neutral.
2. *“Because she’s Caitlyn Jenner. Come to FL sistah where you can run someone over with no consequences.”* — the negativity is implied and sarcastic. There is no clear negative word.
3. *“Tonight may be the night when I finally expose myself to the Gilmore Girls hysteria”* — ironic tone, not a plain complaint.
4. *“Monday was Gucci but then it went downhill”* — the tweet flips mid-sentence, and the model missed the turn.
5. *“Hahaha 2nd place in Miss USA was Miss Texas…”* — sarcasm around a joke. It does not look like a clear negative tweet.

I saved one checkpoint only, after loading the best validation-loss state. When I reloaded it on three held-out sentences, it predicted positive, negative, and neutral in that order, which matched what I intended.

Serving — Table HW4.1

I used the same 32 prompts on `facebook/opt-125m` (a different Hub model from BERT). The first prompt is mine. For a batched vLLM call, per-request latency is the wait for the whole batch, because the requests come back together.

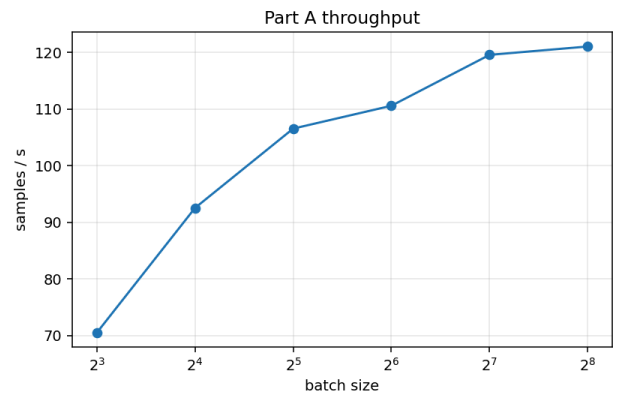
Configuration	Tokens/sec	p50 latency (ms)	p95 latency (ms)
HF pipeline, batch 1	115.7	269.3	280.8
vLLM, batch 1	97.8	319.1	328.4
vLLM, batch 8	627.8	349.2	354.9
vLLM, batch 32	2273.5	383.7	386.5

At batch 1, vLLM was already a bit slower than the Hugging Face pipeline on both tokens/s and p50. At **batch 8**, p50 is still worse than the HF baseline (349 ms vs 269 ms), but tokens/s jumps from about 98 to 628, so total throughput is improving while one person waits longer. Batch 32 keeps that pattern. For an interactive app I care about **p50**, because that is what one user feels. Tokens/s matters when many requests share the GPU.

Part A — batch scaling

I kept the HW4.a fine-tune settings and doubled the batch until a single step ran out of memory. Each size that fit also got one pass over the 3,500 training rows.

Batch	Samples/s	Peak GB	Seconds / epoch
8	70.5	2.09	49.7
16	92.5	2.49	37.8
32	106.5	3.76	32.9
64	110.6	6.15	31.7
128	119.6	11.00	29.3
256	121.0	20.65	28.9
512	OOM	—	—



Part A throughput

The largest batch that ran was **256**. The fastest was also 256, at 121.0 samples/s. By batch **128** I was already within 5% of that peak, so the useful gains stop before the memory wall. Batch 512 is the first OOM.

I only had one GPU. The October 23 note says one GPU is enough, so I do not have a second-card plot or a measured max-batch ratio. If I had two cards, that ratio still would not equal the VRAM ratio. Weights, Adam state, and the CUDA context are a fixed cost on every card. Only activation memory really scales with batch, and a power-of-two ladder skips sizes between the last success and the first OOM.

Part B — memory techniques

I tried each technique alone, then the combination the ranking rule picked. Table HW4.5.1 is for this one card (L4):

Table HW4.5.1 — Report this table once per card

Technique	Max batch	Throughput (samp/s)	Peak memory (GB)	Eval accuracy
Baseline (HW4 settings)	256	92.8	2.48	0.7267
Mixed precision	256	189.3	2.26	0.7040
Gradient checkpointing	1024	70.7	2.09	0.7267
Gradient accumulation	128 (physical 8)	99.5	2.32	0.6520
8-bit optimiser	256	99.3	1.90	0.7187
Best combination	256	222.0	1.64	0.7160

Gradient checkpointing cut throughput from 92.8 to 70.7 samples/s (**–23.8%**) and only saved about 0.39 GB at batch 16. That is the compute-for-memory trade. It did raise the max batch to 1024.

Gradient accumulation used physical batch 8 × 16 steps, so the effective batch is 128. Accuracy was 0.6520. Time to the same number of optimiser steps as the baseline was **816.9 s**, versus **113.1 s** for the baseline fine-tune. The lower peak here is mostly the smaller physical batch.

I ranked the techniques by memory saved per unit of throughput lost at the batch-16 runs:

Rank	Technique	Saved GB	Δ samp/s	Saved GB / samp/s lost
1	8-bit optimiser	0.58	+6.5	free — apply first
2	Mixed precision	0.22	+96.5	free
3	Gradient accumulation	0.16	+6.7	free
4	Gradient checkpointing	0.39	–22.1	0.018

I would apply **8-bit AdamW** first. On a 32 GB card I would **not** bother with gradient checkpointing for this BERT run. It saves under 1 GB at batch 16 and is the only technique that clearly slows me down. The combination I kept was AMP + 8-bit AdamW: 222.0 samples/s, 1.64 GB, test accuracy 0.7160.

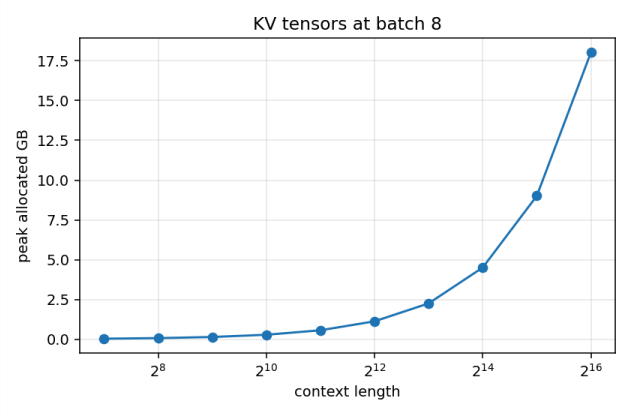
Part C

Part C skipped (optional).

Part D — serving again, and KV cache

I re-ran the vLLM table on the same L4. Batch 1 / 8 / 32 gave 97.9 / 622.9 / 2354.5 tokens/s. That is almost the same as Table HW4.1. Hardware, precision, model, and prompts did not change, so I am not treating the small differences as a measurement flaw.

I also allocated the key and value tensors for `facebook/opt-125m` at batch 8 across context lengths. That number is the tensor size, not a full vLLM serve.



KV cache

Context	Peak GB
128	0.05
256	0.09
512	0.16
1024	0.30
2048	0.58

Context	Peak GB
4096	1.14
8192	2.27
16384	4.52
32768	9.02
65536	18.02

The longest context at batch 8 that still fit was **65536**. I did not hit an allocation OOM on this ladder. OPT-125m's KV tensors are small, so that wall sits past the 2048 position limit I used in the vLLM serve.

Part E — analysis and summary

One full fine-tune on this L4, using the baseline wall-clock:

GPU-hours = 113.1 / 3600 = **0.0314**

That is training seconds only, not how long the Colab tab was open.

The single configuration I would use on this hardware is the **best combination** row: AMP + 8-bit AdamW. Peak memory **1.64 GB**, throughput **222.0** samples/s, wall-clock **47.3 s**, accuracy **0.7160**. It is the fastest row in Table HW4.5.1 that stays close to the baseline accuracy (0.7267), and I do not need checkpointing when batch 256 already fits.

I recorded that same summary, and Table HW4.5.1 above, in `METRICS.md`. This is the HW4.5 summary. It is not an HW5 baseline.

Conclusion

On this L4, BERT-base at batch 16 is not memory-bound (2.48 GB). Throughput stops improving much by batch 128, and the card OOMs at 512. Mixed precision and 8-bit AdamW are the useful changes for me. Checkpointing is the wrong first tool unless I need a much larger batch. On the serving side, larger vLLM batches raise tokens/s while p50 gets worse than the HF batch-1 baseline, so for a chat-style demo I would watch latency, not only throughput.